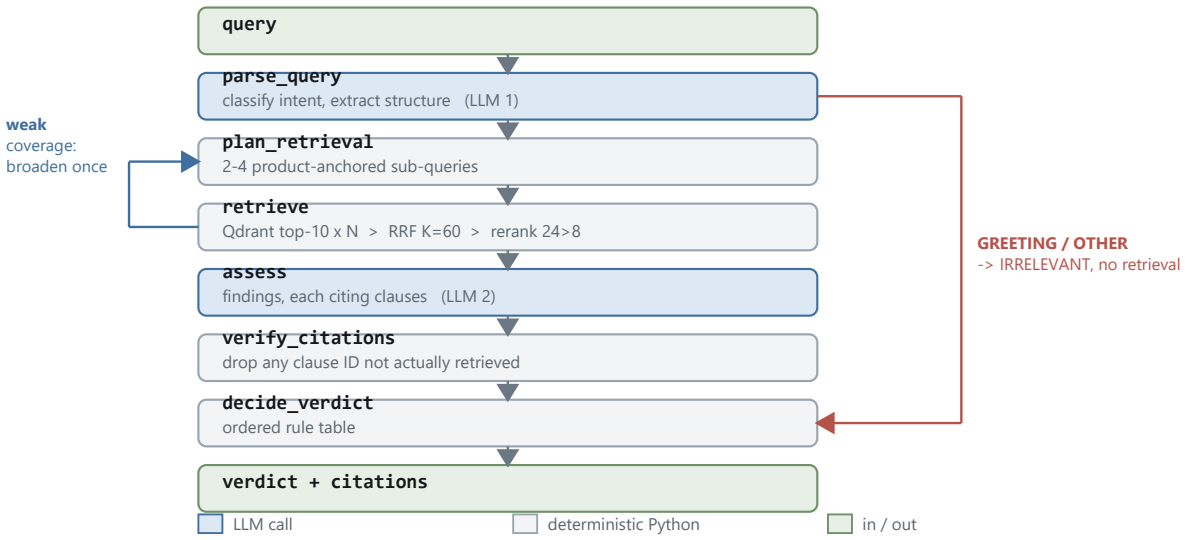
Sharia Compliance Agent

Technical design, production evolution, and what was left undone.

Status. Running against the real AAOIFI *Shari'ah Standards* — 1,264 pages, 54 standards, 1,973 normative clauses. LangGraph agent, deterministic verdict layer, REST API, trace observability, public deployment. 185 offline tests; a 66-case retrieval eval and a 45-case verdict eval, both measured. Anything unbuilt is marked *(not built)*.



1. Architecture decisions

The LLM produces findings; plain Python produces the verdict. The model identifies issues and cites clauses. It is never asked for the label. Aggregation is an ordered rule table in `app/agent/verdict.py`, so identical findings always yield an identical verdict and every rule is unit-testable.

This follows from an asymmetry specific to compliance: a false `COMPLIANT` ships a prohibited product; a false `NEEDS_REVIEW` costs a reviewer an hour. Ambiguity therefore resolves toward `NEEDS_REVIEW` — in code, not in a prompt. §3 shows both sides of that bargain: zero false approvals, and three plainly permissible products referred to a human anyway.

Two conditional edges make this a graph rather than a chain: an **intent router** after `parse_query`, and a **broaden-and-retry** edge after weak retrieval. A greeting or off-topic message returns `IRRELEVANT` having retrieved nothing — a fourth outcome added deliberately, since `NEEDS_REVIEW` is a human work queue and greetings in it both waste a reviewer and corrupt the `NEEDS_REVIEW` rate §3 uses as a degeneracy alarm.

The corpus is the published standard, not a synthesis. An earlier iteration ran on markdown modelled on the standards. It worked, and it was a trap: paraphrased rules cite documents that exist nowhere, making the central claim — *a verdict is traceable to an authority* — unfalsifiable.

Component	Chosen	Why	Ruled out
Vector store	Qdrant Cloud	Index survives outside the container, so the corpus is re-ingested without a redeploy. 1,973 × 3072 × 4B ≈ 24 MB.	**Chroma** — index ships in the image, so a corpus update means a redeploy. **pgvector** — right answer inside an existing Postgres estate; rejected as an extra database to operate. **Pinecone** — proprietary, no standard export. **numpy flat index** — adequate today; rejected because the corpus grows and the index must outlive the container.
Embeddings	`text-embedding-3-large`, 3072-d	One key, one bill, one SDK shared with chat. Whole corpus ≈ $0.02.	**Local ONNX MiniLM** — the original choice; once embeddings became an API call it was ~110 MB resident for worse retrieval. **sentence-transformers** — ~2 GB. `3-small` — retrieval quality bounds verdict quality and the delta costs two cents.
Framework	LangGraph	Typed state, two conditional edges, node logic that stays ordinary readable Python.	**LangChain agent executors** — opaque control flow. **Hand-rolled state machine** — LangGraph is that abstraction, already tested.
Chunking	One AAOIFI clause, prefixed with its full heading lineage	The drafters numbered every rule; those boundaries are the citation unit.	**512-token windows** — cut rules in half. **Section-level** — too coarse to cite. **Clause without prefix** — near-identical embeddings. **LLM-generated blurbs** — ~2,000 ingest calls and a non-reproducible index, to synthesise context the document already states.

Without that prefix AAOIFI clauses embed almost identically — the register is elliptical, and dozens of rules open "*It is permissible for the Institution to…*". This is *clause-level chunking with hierarchical contextual headers*, not Anthropic's Contextual Retrieval: ours reads the document's own hierarchy rather than generating a blurb per chunk, so it is deterministic and costs no inference.

Retrieval is four stages, each fixing a measured failure. Product-anchored sub-queries (a savings-account question no longer retrieves the general riba prohibition instead of the deposit rules) → vector search at top-10 each → **RRF fusion**, K=60, since scores from different sub-queries are not on a common scale and averaging them is arithmetic on incomparable numbers → **cross-encoder rerank**, 24 to 8. Rerank failure degrades to the fused order; it is never a dependency.

Configuration	Recall	MRR	Latency
top_k=5, no rerank	0.955	0.898	0.72s
top_k=10, no rerank	0.955	0.898	0.66s
top_k=5, rerank	0.955	0.895	1.18s
top_k=10, rerank	**1.000**	**0.924**	1.26s

Neither helps alone: at k=5 the governing clause never enters the pool, so the reranker cannot promote it; at k=10 without reranking it enters but sits below the cutoff. Shipped separately, either looks like a no-op — the clearest argument here for building the eval set before tuning.

Two model findings. Provider routing changes latency 8× (15.6 → 130.3 tok/s sorted by throughput; §5 Risk 1 is the cost). And reasoning effort changes *correctness*: at `high` the model scored 3/4 against 4/4 at `low`, inventing `CONDITIONAL` findings the query never raised — given more budget it manufactures doubt, which is corrosive where the rules already lean to `NEEDS_REVIEW`. Together these took an assessment from 151s to 9–25s.

2. Scaling to 50,000 queries/day

50,000/day is 0.58 req/s averaged, but compliance work is bursty: at 80% within 8 hours, mean load is ~1.4/s peaking at 4–5/s, and at 9–25s per assessment that is **~100 concurrent in-flight requests** — the number that breaks things, not the daily total.

Component	Daily	Cost
Prompt tokens	125M	$8.13
Completion tokens	100M	$18.00
Query embeddings	5M	$0.65
Reranking *(estimated)*	215M	~$10.75
Total		**≈ $1,150/month**

Completion tokens dominate, so reasoning effort is the main cost lever as well as the main quality lever. Reranking — bought for +0.54s and +0.045 recall — is ~28% of the bill: cheap in latency, not in money.

What breaks first, in order. (1) The host: one 512 MB free instance that sleeps and cold-starts in ~50s. (2) Synchronous handling — each request holds a worker for 9–25s, so ~100 concurrent exhausts any sane pool long before CPU matters. (3) No shared cache: caching is per-process, useless behind more than one instance, and compliance queries repeat heavily. (4) Qdrant free tier — storage is a non-issue (24 MB of 4 GB), 0.5 shared vCPU at ~20 searches/s is not. (5) Single vendor on two paths: chat, embeddings *and* reranking all traverse OpenRouter, whose rerank endpoint is undocumented. (6) In-process traces, lost on restart.

What I would change. Convert to `async` end to end so a worker awaits rather than blocks, plus a queued mode — `POST /assess` returns `202` with a trace ID, workers consume a queue, the client polls or takes a webhook; synchronous mode stays for interactive use behind a hard timeout. Scale horizontally on stateless containers, autoscaled on **queue depth, not CPU**, which is near-idle while awaiting the LLM. Cache in two Redis layers — exact-match on the normalised query and a semantic cache on its embedding at ≥0.97 — keyed on corpus version, prompt version and model, so a stale answer cannot outlive the change that invalidated it. Make reranking conditional on a close fused ranking, the largest cost lever after reasoning effort. Move Qdrant to a dedicated cluster with replicas and scalar quantization. Pin a provider allowlist, add a circuit breaker, cache the ~2,100-token static prompt, and contract directly with the rerank vendor. Version everything that affects output; without it a regression cannot be bisected.

Already done: batched sub-query embeddings. Four sequential round trips became one — 1,925 ms → 594 ms, taking `retrieve()` from ~3,271 ms to ~1,940 ms. It saves no money (embeddings bill per token), so the win is latency, rate-limit headroom and one failure surface instead of four. It falls back to individual calls when the batch raises **or returns the wrong number of vectors** — the dangerous case, since a truncated response would zip vectors against the wrong sub-queries and silently retrieve for text nobody asked about.

3. Evaluation & quality

A wrong verdict has three causes with three different fixes: **retrieval failure** (the clause never arrived), **reasoning failure** (it arrived and was misapplied), **rule failure** (findings were right, aggregation mis-fired). End-to-end accuracy cannot separate them, so each is measured alone. The architecture makes that tractable: the verdict is deterministic given findings, so rule failure is fully unit-testable, and retrieval is measurable with no LLM at all.

Retrieval — 66 cases, no LLM in the loop. Recall 0.985, MRR 0.890, out-of-scope top score 0.255 against a 0.35 coverage threshold; per-kind because an aggregate hides what matters — `coverage` 1.000 (50), `adversarial` 1.000 (8), `near_miss` 0.875 (8). The standing near-miss is instructive: "*what conditions apply to a Salam contract*" retrieves SS-10, the same transaction in business language retrieves none, and production survives only because `parse_query` labels it "salam sale". So **retrieval is carried by the LLM's contract-type labelling, not the description**, making a parse error a *silent* retrieval failure. Kept failing as the witness.

Verdict — 45 cases, five strata, whole graph. Each in-scope case names a governing clause verified against the published edition; two references were wrong when written and were caught by that check. Borderline cases accept more than one verdict on purpose — insisting on one answer where a reviewer could land in two places measures luck. A point is awarded per criterion satisfied:

Criterion	What it rewards	Score
`grounded`	Cited a governing standard	**0.975**
`verdict`	A verdict a reviewer would accept	**0.933**
`named it`	Named the prohibition, not just declined to approve	**0.900**
`decided`	Approved a clear permission	**0.700**
`deferred`	Left a borderline case open	**0.700**
	Overall 118/130	**0.908**

False `COMPLIANT`: 0/20, reported separately and never tradeable against points earned elsewhere. But that number is weaker than it looks, and saying so matters more than the number: 20 cases is small enough that zero failures is still consistent with a true rate of **13.9%** (rule of three) — ~7,000 bad approvals a day at target volume. It is also saturated, so it can only detect regression; the cases are textbook single-clause prohibitions, not the compound structures real proposals use; and it was authored by the person who built the system. The informative number is **decisiveness at 0.700**: the agent is reliable about what it must not approve and much weaker at committing when committing is right.

Still needed. A scholar-authored set of ~300 cases — `SCHOLAR_REVIEWED = False` prints on every report so no run is quoted as evidence of Sharia correctness by accident. A `parse_query` accuracy set, which the Salam miss showed nothing covers. Extraction assertions at ingest, since the citation bug was caught by human suspicion, which does not scale. A regression set fed by production disagreements.

Metrics to track: false-`COMPLIANT` rate (target ≈0, each occurrence an incident); adversarial recall; citation precision; groundedness by LLM judge with human audit; verdict stability — measured at **2× `COMPLIANT` and 3× `NEEDS_REVIEW` on five identical runs**, and visible again across eval runs; and the `NEEDS_REVIEW` rate as the degeneracy alarm.

Human-in-the-loop. Shadow mode first, output invisible until agreement with reviewers is measured rather than assumed. `NEEDS_REVIEW` goes to a human by definition; 100% of `NON_COMPLIANT` is reviewed initially, since a wrong prohibition blocks legitimate business; and `COMPLIANT` **is sampled continuously**, never dropping to zero, because that is where the costly error hides. Reviewer agree/disagree with a reason is the primary quality signal, feeding a regression set. Quarterly scholar audit; re-validation on AAOIFI revisions. Any change to prompt, model, chunking, corpus or retrieval runs the full suite in CI, with a false-`COMPLIANT` regression blocking outright.

---

4. Observability & debugging

Built: a trace ID minted or adopted per request and propagated by `contextvar`, so every line carries it; structured JSON logs holding sub-queries, retrieved chunk IDs and scores, the resolved prompt and raw completion, the model served, token counts, findings, rejected citations and the verdict rule that fired; `GET /traces/{id}`; and a `/health` returning 503 rather than 200 when Qdrant is unreachable. The prompt is logged **before** the call, so a request that times out still leaves what was sent.

Where logs land matters: on Render, stdout only, and both the log file and the trace store are wiped on every restart. **Nothing in the deployed system retains an assessment record** — the audit store below is not an optimisation but the first thing a regulated deployment needs. One lesson: the test suite once logged to the production path, leaving 3,966 lines across 626 process starts with exactly *one* genuine assessment among them. Telemetry that mixes real and test traffic is confidently misleading rather than merely absent.

Production stack. OpenTelemetry, one span per node, carrying retrieved chunk IDs and scores, prompt and corpus version, model *and provider* served, whether rerank fired or degraded, and the verdict rule. The **audit record is separate from telemetry**: telemetry is sampled and short-lived, the compliance record is not — append-only and tamper-evident, holding query, retrieved clauses, exact prompts, completion, findings, verdict, rule, reviewer action and authenticated principal, with CBUAE-compliant retention. Alert on leading indicators: `NEEDS_REVIEW` rate rising (drift), citation rejection rising (hallucination), retrieval scores shifting (corpus or embedding), rerank fallback rising (the undocumented endpoint changed), verdict distribution shifting with no deploy (upstream model changed) — the last is undebuggable unless the provider served is recorded per call.

Debugging a wrong verdict is a decision procedure, not an investigation. Pull the trace by ID. Was the governing clause retrieved? If not it is a retrieval failure — check sub-queries, fused scores, whether rerank or the broaden retry fired; reproducible offline with no LLM and directly addable to the golden set. If it was, were the findings right? If not it is a reasoning failure, and the exact prompt and completion are in the trace to replay. Otherwise it is a rule failure, which is deterministic: add the findings to the rule table as a unit test. Add the case to the regression set either way. A **replay harness** *(not built)* would make this cheap corpus-wide, since the verdict is a pure function of findings and findings a pure function of prompt plus chunks.

Trace *content* is what makes this work. The `reasoning=high` misdiagnosis looked like "the model is bad at JSON" until token-level inspection showed 3,072 of 3,072 completion tokens spent on reasoning; the first retrieval failure looked like bad reasoning until the trace showed the sub-query was literally `"riba"`. Traces must record token composition and sub-queries, not just totals and the question.

5. Security & regulatory compliance

Risk 1 — Sensitive queries leave the jurisdiction to unvetted providers. *Critical.* A query may carry client names, deal terms or material non-public information. It goes to OpenRouter, which routes to any of 29 upstreams in unknown jurisdictions — and the throughput-sorted routing adopted for latency makes that selection **non-deterministic**, so consecutive calls can be served by different companies. Reranking widens it: the query *and* 24 clauses reach a second upstream through an undocumented proxy. Incompatible with CBUAE outsourcing and data residency expectations and with PDPL cross-border rules. *Mitigate:* provider allowlist with `data_collection: "deny"` and `allow_fallbacks: false`, restricted to providers under a DPA; preferably region-resident (Azure OpenAI UAE North, Bedrock me-central-1, or self-hosted); PII/MNPI redaction before egress; log the provider served so residency is auditable.

Risk 2 — No authentication or authorisation. *Critical.* `/assess` is public and unauthenticated — the brief required a URL reachable without sign-in, directly in tension with this control — and there is no notion of *who* asked, which defeats the audit trail a regulator expects. *Mitigate:* OAuth2/OIDC against the bank IdP with short-lived tokens; mTLS service-to-service; RBAC separating assessors from sign-off reviewers; per-tenant rate limits; WAF and IP allowlisting. Demonstrator and internal deployment should be separate environments, not one service behind a flag.

Risk 3 — Sensitive query text in plaintext logs. *High.* `LOG_PROMPTS=true` is the current default and writes the resolved prompt, including the customer's query, to stdout and disk — in production, into log aggregation retained under the log platform's policy rather than the bank's. It is on here because reproducibility beat confidentiality in a demonstrator; that trade inverts in production. *Mitigate:* default it off; separate the encrypted audit record from redacted telemetry; field-level encryption for query text; retention honouring PDPL data-subject rights.

Risk 4 — Secrets lifecycle. *High.* Credentials sit in `.env` and a host dashboard with no rotation, per-environment separation or revocation path. During this project a live key was pasted into a chat transcript and had to be rotated — a realistic failure, not a contrived one. *Mitigate:* managed secret store with automatic rotation, workload identity over static keys, per-environment credentials, secret scanning in CI and pre-commit.

Risk 5 — Presenting model output as a ruling. *High, and regulatory rather than technical.* If a verdict is perceived as a ruling, the bank has delegated Sharia authority to software, which no regulator or Sharia Supervisory Board would accept. The interface sharpens it: a confidence of 0.934 beside `NON_COMPLIANT` reads as certainty when it is a coarse blend of citation count and retrieval strength. *Mitigate:* UI language making every output advisory; mandatory human approval before a verdict affects a product decision; the SSB retains formal authority with the tool recorded as an input; confidence as a qualitative band or dropped. The design supports this — `NEEDS_REVIEW` is first-class — but the control is organisational.

Also live: the AAOIFI standards are licensed. Both PDFs were removed and git-ignored, but git history still holds them and Qdrant payloads still hold verbatim clause text. History rewrite and authenticated-only cluster access are the fixes.

6. What I cut

- **Scholar review of the verdict set.** The expectations are mine, so a run evidences consistency with my reading of the standards rather than Sharia correctness — and the ten borderline cases are where a scholar would disagree. **Still what I would do next.**
- **Extraction assertions and `tests/test_aaoifi.py`.** The most intricate module — the one that produced a silent citation-corruption bug — is exercised only indirectly, so the next regression is found the way the last one was: a human noticing an implausible number.
- `parse_query` **accuracy unmeasured**, so a parse error is a silent retrieval failure invisible to both evals.
- **No caching, async or auth:** cannot survive production load, cannot be pointed at real client data.
- **Durable traces and replay harness** — the procedure above works for a live incident, not a historical one.
- **Arabic edition not ingested.** Arabic is the authoritative AAOIFI text, so the system reasons from a translation. Mitigated architecturally: `lang` is a payload field and numbering is identical across editions, making it a filter and a re-ingest rather than a redesign.
- **Free-tier hosting** — ~50s cold start and failure under real concurrency. Deliberate: the brief asked for a public URL, not a production deployment.